

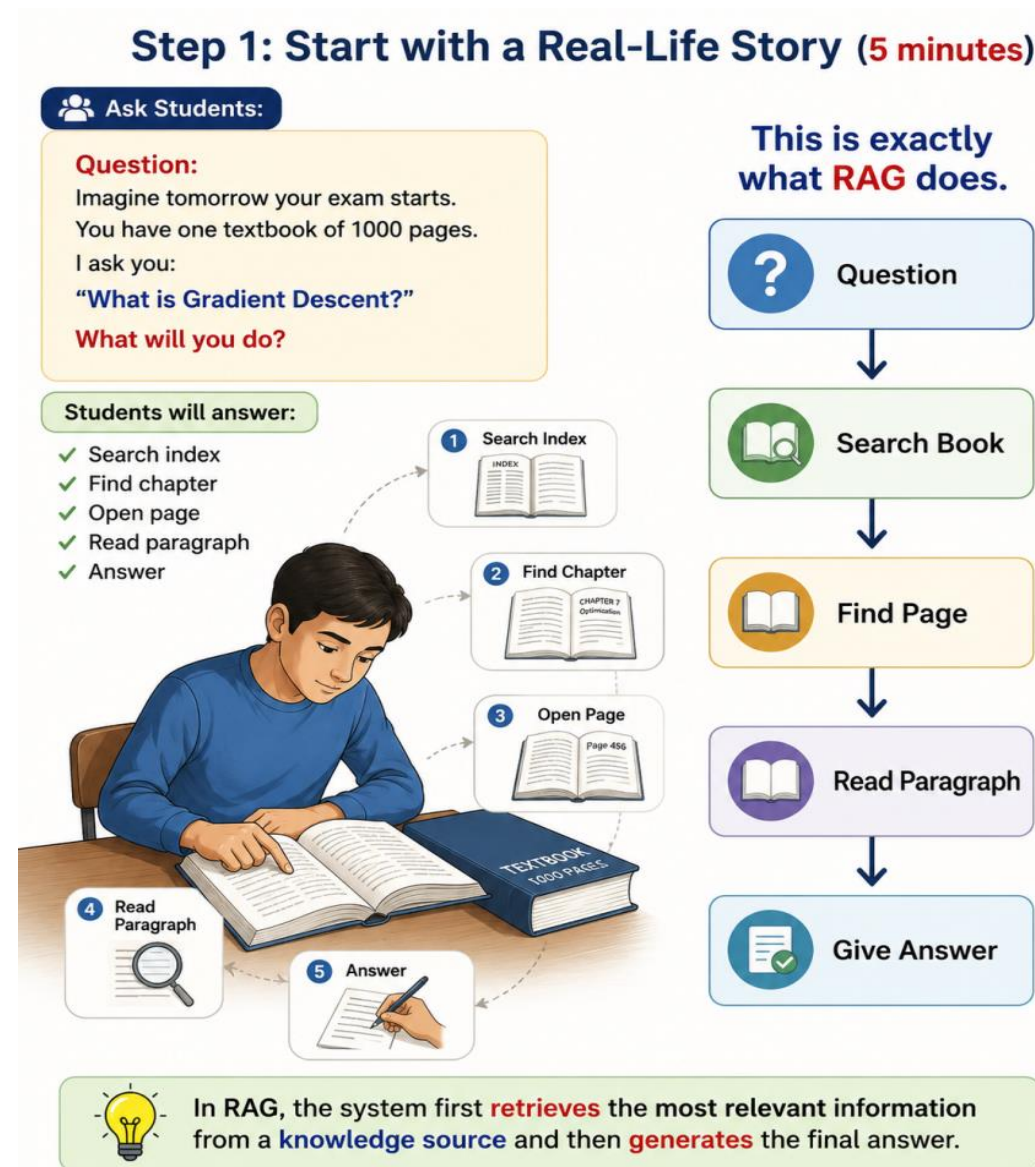
Retrieval-Augmented Generation (RAG)

Dr. A. V. Brahmane

Department of Computer Engineering

Sanjivani College of Engineering, Kopergaon

Introduction to Retrieval-Augmented Generation (RAG)



Step 2: Problem with ChatGPT Alone



ASK:

If I ask ChatGPT

Who is Principal of
our college?



Will ChatGPT know?



ChatGPT



Students: "No"



Why?

- It was trained earlier
- It doesn't know today's information
- It cannot read your college documents



Explain:

LLMs only know what they **learned during training**.



They don't automatically know

Today's News



Events happening
after training

**Company
Documents**



Internal reports,
policies, data

Your PDF



Syllabi, notes,
files you upload

**Your College
Website**



Notices, updates,
announcements

**Hospital
Records**



Patient data, reports,
prescriptions



So, ChatGPT alone may give **wrong, outdated**, or **general** answers
because it cannot access your **real-time or private information**.

Step 3: Why RAG?



LLM Memory = Old Knowledge

An LLM only knows what it learned during training.



Can we give LLM our documents before answering?

Students:
"Yes!"



Exactly! That idea is called

Retrieval-Augmented Generation (RAG)

LLM Alone (No retrieval)



Q: What is admission process of our college?



LLM may give a general or incorrect answer because it does not have your documents.

Uses only training data (old knowledge)



RAG (With your documents)



Your Documents (Knowledge Base)

- College website
- PDF / Notes
- Policies / Reports
- Latest information



Q: What is admission process of our college?



RAG finds the relevant information from your documents and gives an accurate answer.

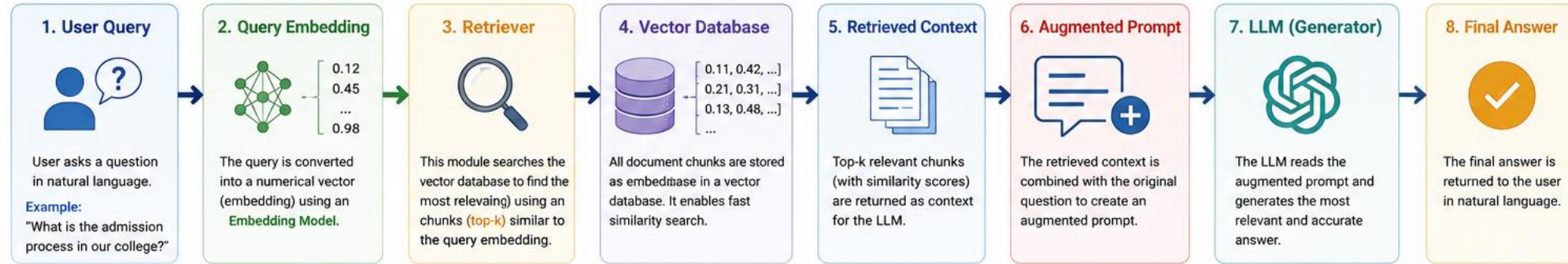


So, RAG gives the LLM **your own, latest and organization-specific** information before it generates the final answer.

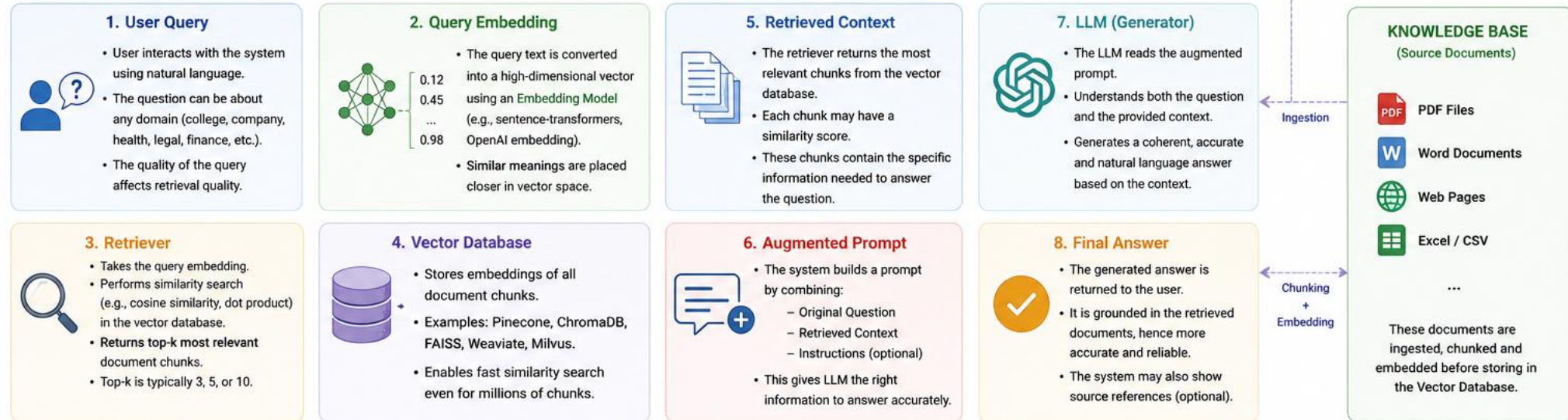
RAG ARCHITECTURE

(Retrieval-Augmented Generation)

RAG combines information retrieval from external knowledge sources with the generation power of LLMs to produce accurate, up-to-date and context-aware answers.



DETAILS OF EACH BLOCK



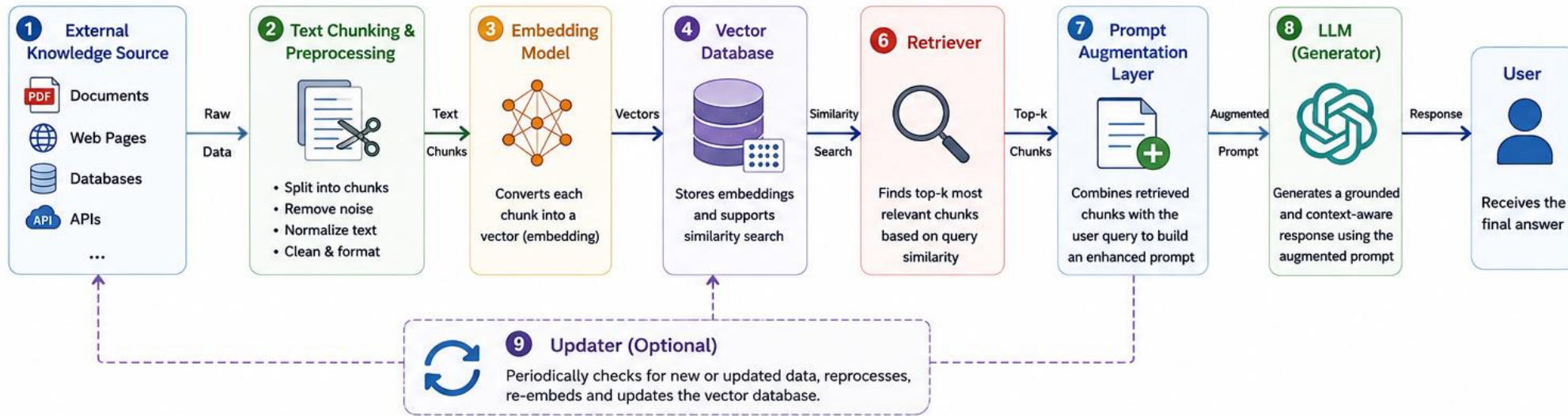
In short: RAG = **Retrieve** (find relevant information) + **Augment** (add it to the question) + **Generate** (LLM produces the answer)

What is RAG ?

- Retrieval-Augmented Generation (RAG) is a way to make AI answers more reliable by combining searching for relevant information and then generating a response.
- Instead of guessing based only on old training data, it first finds useful data from external sources (like documents or databases) and then uses it to give a better answer.
- Fetches up-to-date data and reduces incorrect or made-up answers
- Works well with specialized data like medical or legal content
- No need to retrain the model every time new data comes in
- Can use user-specific data to give more relevant responses

Components of RAG

RAG combines retrieval of relevant information with LLM generation to produce accurate, context-aware and up-to-date answers.



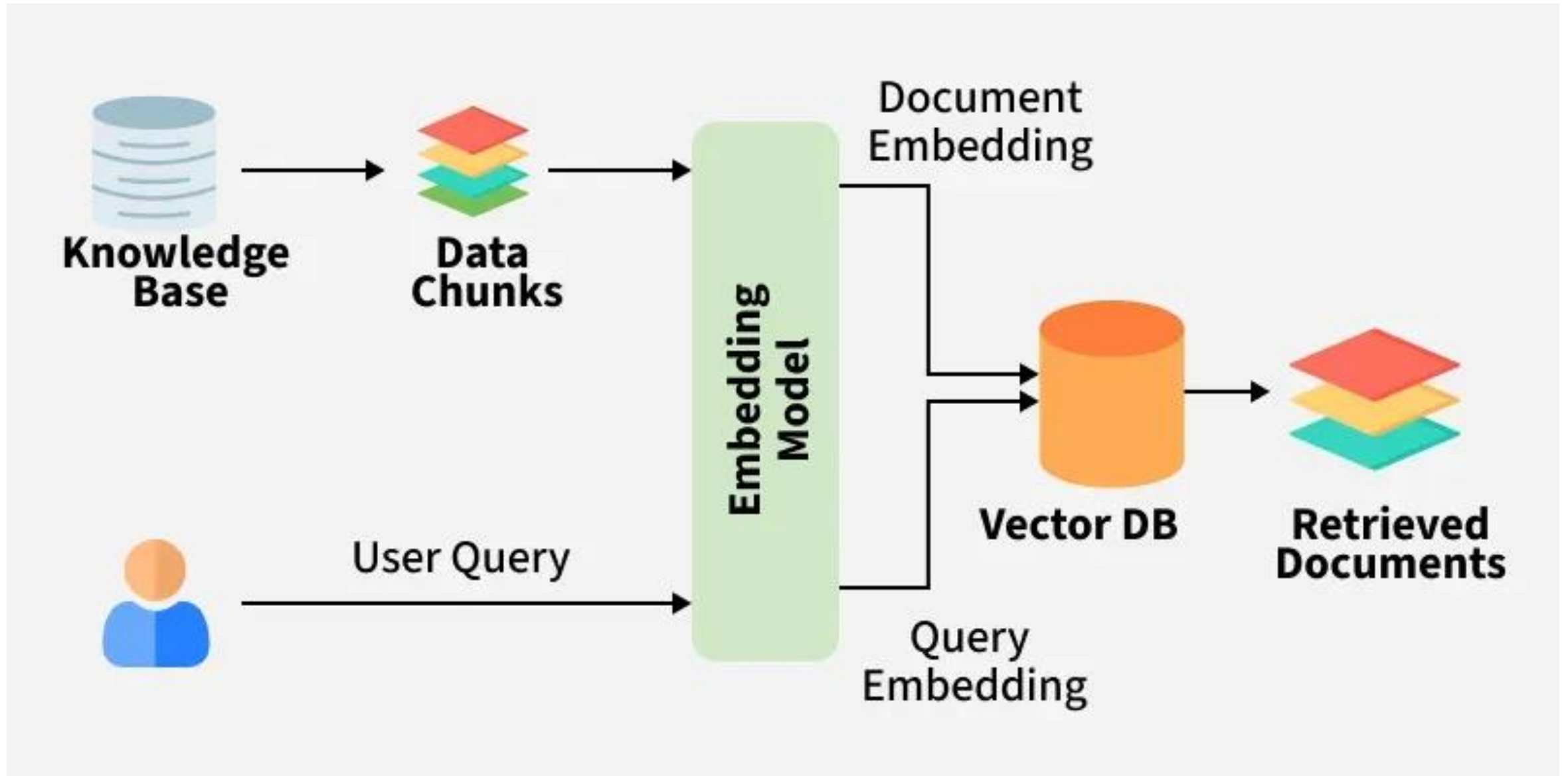
COMPONENTS EXPLAINED

1  External Knowledge Source Stores domain specific or general information like documents, APIs or databases.	6  Retriever Finds and returns the most relevant chunks from the database based on query similarity.
2  Text Chunking and Preprocessing Breaks large text into smaller, manageable chunks and cleans it for consistency.	7  Prompt Augmentation Layer Combines retrieved chunks with the user's query to provide context to the LLM.
3  Embedding Model Converts text into numerical vectors that capture semantic meaning.	8  LLM (Generator) Generates a grounded response using both the query and retrieved knowledge.
4  Vector Database Stores embeddings and enables similarity search for fast information retrieval.	9  Updater (Optional) Regularly refreshes and re-embeds data to keep the knowledge base up to date.
5  Query Encoder Transforms the user's query into a vector for comparison with stored embeddings.	



How it works: The system retrieves relevant information from an external knowledge source, converts it into vectors, stores it in a vector database, and uses it to augment the user query. The LLM then generates accurate and contextually relevant answers.

Working of RAG



Working of RAG

- The system first searches external sources for relevant information based on the user's query instead of relying only on existing training data.
- **Creating External Data:** External data from APIs, databases or documents is chunked, converted into embeddings and stored in a vector database to build a knowledge library.
- **Retrieving Relevant Information:** User queries are converted into vectors and matched against stored embeddings to fetch the most relevant data ensuring accurate responses.
- **Augmenting the LLM Prompt:** Retrieved content is added to the user's query giving the LLM extra context to work with.
- **Answer Generation:** LLM uses both the query and retrieved data to generate a factually accurate, context aware response.
- **Keeping Data Updated:** External data and embeddings are refreshed regularly in real time or scheduled so the system always retrieves latest information.

What is Knowledge Base?

Knowledge Base means

- Collection of documents
- Examples
- PDF
- Word
- Excel
- Website
- Books
- Emails
- Reports
- Everything becomes knowledge.

Example: College Admission Chatbot

- Suppose your college has the following files:
- College_Data/
 - |— Admission_Guidelines.pdf
 - |— Fee_Structure.xlsx
 - |— Hostel_Rules.docx
 - |— AI_Club_Report.pdf
 - |— Academic_Calendar.pdf
 - |— Principal_Message.docx
 - |— Library_Rules.pdf
 - |— Placement_Statistics.xlsx
 - |— www.sanjivani.edu.in
 - |— NAAC_Report.pdf
- These files together form the **Knowledge Base**.
- Think of it as a **digital library**.

Documents are Collected

- The RAG system is given a folder containing documents.
- Knowledge Base
- ↓
- Admission.pdf
- Fee.xlsx
- Hostel.docx
- Library.pdf
- Website
- Reports
- Emails
- At this stage, they are just ordinary files.

STEP 2

RAG Reads Every Document

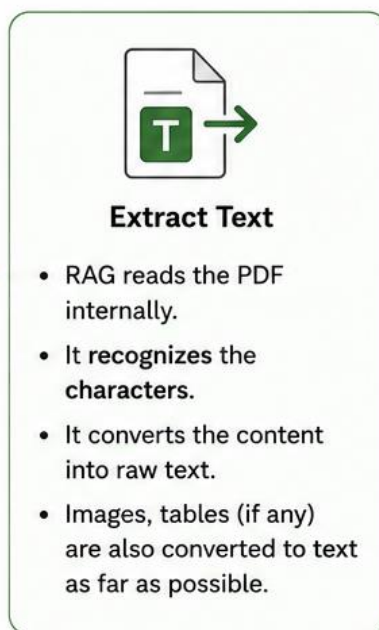
RAG first reads each document in the knowledge base and extracts the text content from it.

Suppose Admission.pdf contains

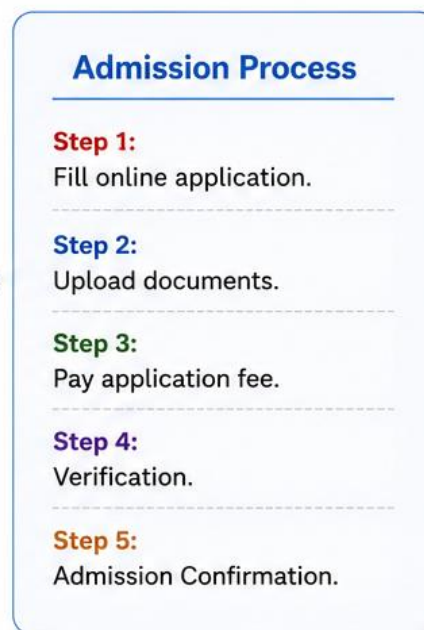


This is the original PDF file stored in your computer.

RAG extracts the text



Extracted Text (Raw)



This is the plain text extracted from the PDF.

Important Point!

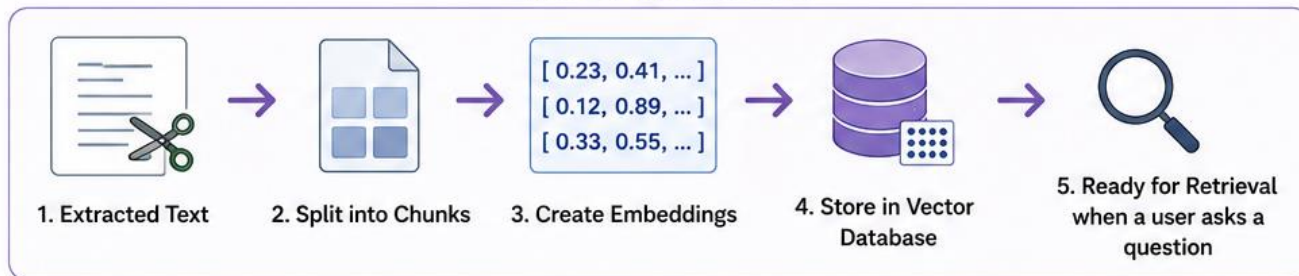


The PDF itself is **NOT** stored in the vector database.



Only the extracted text is used for further processing (Chunking, Embedding and Storage).

What Happens Next?



Key Takeaway

RAG converts unstructured documents (PDF, Word, Excel, etc.) into clean, machine-readable text. This text (not the original file) is used to build the knowledge base that powers accurate retrieval and generation.

STEP 3 Chunking – Splitting Large Document into Small Pieces



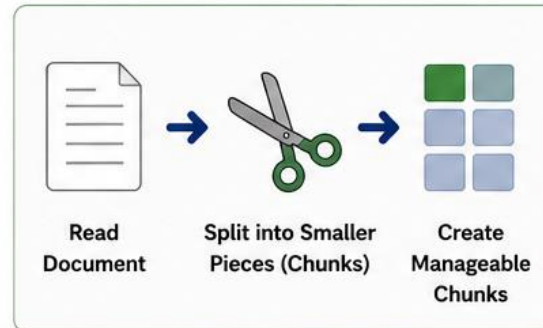
A 100-page PDF is too large for efficient retrieval and for the LLM to process. RAG splits it into smaller pieces called **chunks**.

Original Document (Example)



Too large to send to LLM
in one go!

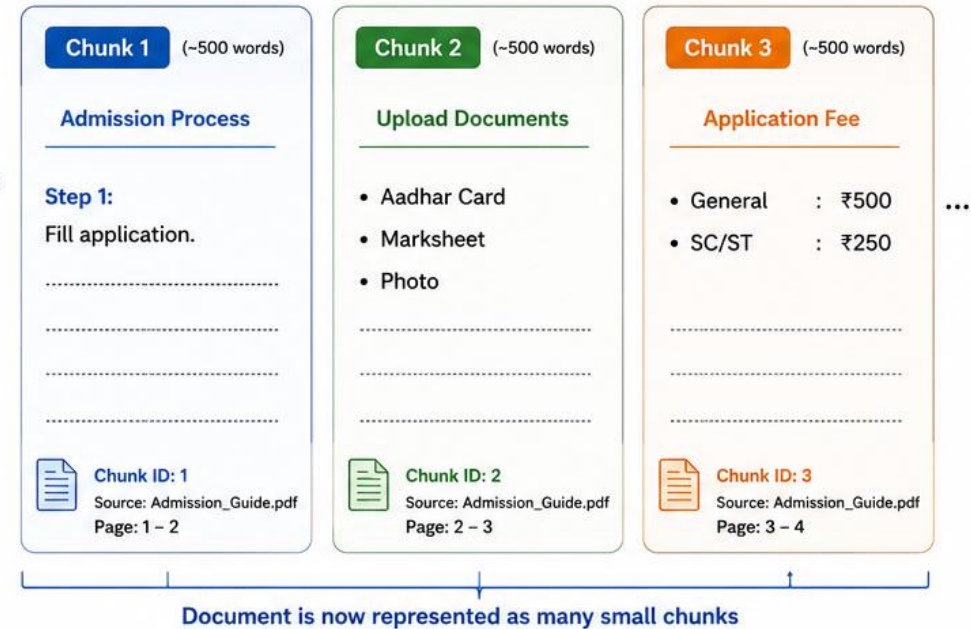
Chunking Process



How Chunking Works?

- RAG breaks the document based on a fixed size (e.g., 500 words) with some overlap (e.g., 50 words).
- Overlap helps to keep context between chunks.
- Each chunk is independent and can be retrieved separately.

Example: Document is split into Chunks



Why Chunking is Important?



Makes retrieval faster and more accurate.



Reduces the amount of data sent to LLM.



Helps LLM focus only on relevant information.



Saves tokens and reduces cost.

Without Chunking



100 Pages

- Slow
- Expensive
- May exceed token limit
- Less accurate

With Chunking



Many Small Chunks

- Fast
- Cost-effective
- Within token limit
- More accurate



Key Takeaway

Instead of one huge document, RAG now has many small, overlapping chunks. These chunks are easy to search, store, and use to answer questions accurately.

Example

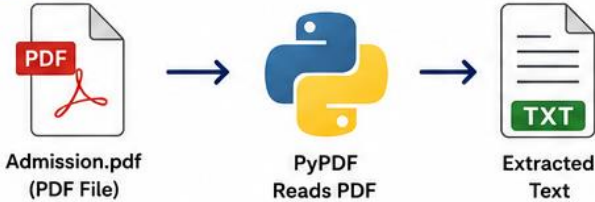
If chunk size = 500 words and overlap = 50 words → A 100-page PDF (~50,000 words) → Will be converted into ~150 to 200 chunks

READING PDF FILES IN PYTHON USING PyPDF



In RAG, the first step is to read PDF documents and extract their text.
Python does not understand PDF directly, so we **extract only the plain text**.

1 READ A SINGLE PDF



Python Code

```
1 from pypdf import PdfReader
2 reader = PdfReader("CollegeData/Admission.pdf")
3
4 text = ""
5
6 for page in reader.pages:
7     text += page.extract_text()
8
9 print(text)
```

Output

Admission Process

Step 1
Fill Online Application

Step 2
Upload Documents

Step 3
Pay Application Fee

Step 4
Verification

Step 5
Admission Confirmation



Notice

Python didn't keep the PDF.
It **extracted only the text**.



What Happened?

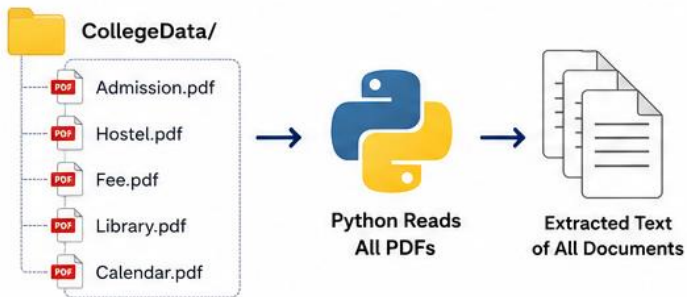
- PdfReader opens the PDF.
- `page.extract_text()` reads the text from each page.
- All text is stored in the variable `text`.
- The PDF file itself is **NOT** stored anywhere.



Key Takeaway

This is the first step in RAG:
Convert PDF → Plain Text

2 READ ALL PDFS IN A FOLDER



Python Code

```
1 import os
2 from pypdf import PdfReader
3
4 documents = []
5
6 folder = "CollegeData"
7
8 for file in os.listdir(folder):
9     if file.endswith(".pdf"):
10         reader = PdfReader(os.path.join(folder, file))
11         text = ""
12         for page in reader.pages:
13             text += page.extract_text()
14         documents.append(text)
15
16 print("Total Documents :", len(documents))
```

Output

Total Documents : 5



What Next?

Now we have the text of all documents in the list documents.
These texts will be chunked in the next step.

How it Works (Summary Flow)



Result



All PDF content is now available as plain text for the next steps (chunking, embedding, storage...).

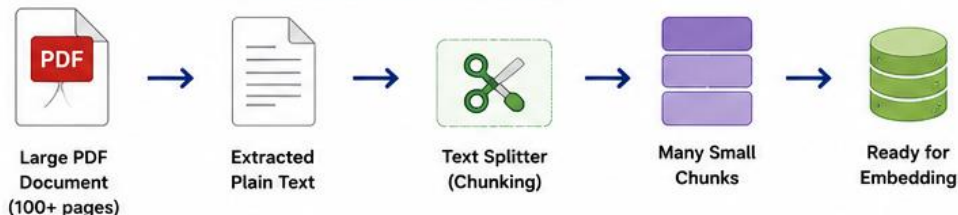
STEP 2: CHUNKING THE DATA

Chunking means splitting a large document into **smaller, meaningful pieces (chunks)**.
This makes it easier to create embeddings and **improves retrieval quality**.

Why Chunking?

- 🎯 Fits within model token limits
- 🔍 Improves retrieval accuracy
- ⌚ Faster and more efficient
- 🧩 Keeps related information together
- 💾 Better embeddings and storage

From Document to Chunks



Chunking Parameters (Example)

- 📄 `chunk_size = 500` → Maximum characters per chunk
- 🔗 `chunk_overlap = 50` → Overlap between chunks
- 🔧 `separators = ["\n\n", "\n", ". ", " "]`
→ Order of priority to split text

What is Overlap and Why?



Overlap = 50 characters

Overlap keeps context from the previous chunk so information is not lost.

PRACTICAL EXAMPLE

1. Extracted Text (from Admission.pdf)

Admission Process

Step 1: Fill online application through our official website. You need to provide basic details like name, contact number, email id, and address.

Step 2: Upload required documents. The mandatory documents are Aadhar Card, 10th Marksheet, 12th Marksheet, and Passport size photo.

Step 3: Pay the application fee online. For General category the fee is Rs. 500 and for SC/ST category the fee is Rs. 250. Fee is non-refundable.

Step 4: Verification of documents will be done by the admission cell. You will receive an email after verification.

Step 5: After document verification and fee payment, admission confirmation will be sent to your registered email id.

2. After Chunking (chunk_size=500, overlap=50)

Chunk 1 (0 - 500 chars)

Admission Process

Step 1: Fill online application through our official website. You need to provide basic details like name, contact number, email id, and address.

Step 2: Upload required documents. The mandatory documents are Aadhar Card, ...

Chunk 2 (450 - 950 chars)

Addhar Card, 10th Marksheet, 12th Marksheet, and Passport size photo.

Step 3: Pay the application fee online. For General category the fee is Rs. 500 and for SC/ST category the fee is Rs. 250. Fee is non-refundable.

Chunk 3 (900 - 1400 chars)

non-refundable.

Step 4: Verification of documents will be done by the admission cell. You will receive an email after verification.

Step 5: After document verification and fee payment, admission confirmation will be sent to your registered email id.

Chunk Info Table

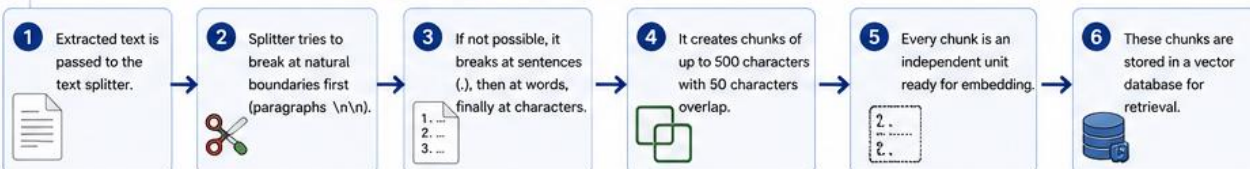
Chunk ID	Start Index	End Index	Length (Chars)
1	0	500	500
2	450	950	500
3	900	1400	500
4	1350	1850	500
...	...	...	...

Each chunk is a small, self-contained piece of text with some overlap to preserve context.

Python Code (Using LangChain)

```
1 from langchain.text_splitter import RecursiveCharacterTextSplitter
2
3 # 1. Create Text Splitter
4 splitter = RecursiveCharacterTextSplitter(
5     chunk_size=500,      # max characters per chunk
6     chunk_overlap=50,    # overlap between chunks
7     separators=["\n\n", "\n", ". ", " "] # split priority
8 )
9
10 # 2. Split the documents
11 chunks = splitter.split_documents(documents)
12
13 # 3. Check the chunks
14 print(f"Total chunks created: {len(chunks)}")
15 print("First chunk preview:\n")
16 print(chunks[0].page_content)
```

How it Works (Behind the Scenes)



Key Takeaway

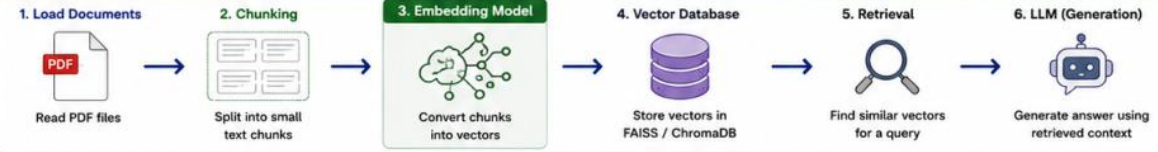
- 💡 Chunking breaks long documents into small, meaningful pieces.
- ✅ Proper chunk size and overlap improve retrieval accuracy.
- ✅ These chunks are the foundation for embeddings and RAG.

STEP 3: IMPLEMENTING THE EMBEDDING MODEL (CONVERT CHUNKS INTO VECTORS)



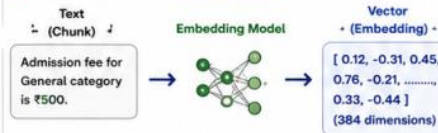
An embedding model converts each text chunk (words) into a list of numbers (vector) that captures its **meaning**. Computers understand numbers, not text. These vectors are used for **semantic search** in RAG.

WHERE WE ARE IN RAG PIPELINE



WHAT IS AN EMBEDDING?

- ✓ A vector (list of numbers) that represents the meaning of text.
- ✓ Similar meanings → similar vectors (closer in space).
- ✓ Different meanings → distant vectors.
- ✓ These vectors enable **semantic search**.



EXAMPLE: CHUNK → EMBEDDING (VECTOR)



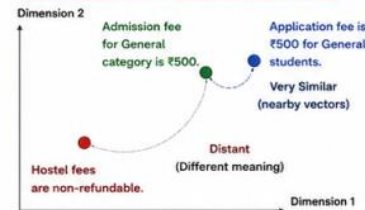
* The above vectors are example values only (not actual).

PYTHON CODE (LangChain)

```
1 from langchain_huggingface import HuggingFaceEmbeddings
2
3 # 1. Load the embedding model
4 embedding = HuggingFaceEmbeddings(
5     model_name="all-MiniLM-L6-v2" # 384-dimensional vectors
6 )
7
8 # 2. Convert a single chunk into embedding (vector)
9 vector = embedding.embed_query("Admission fee for General category is ₹500.")
10 print(len(vector)) # Output: 384
11 print(vector[:10]) # First 10 values (example)
12
13 # 3. Convert all chunks into embeddings
14 vectors = embedding.embed_documents([chunk.page_content for chunk in chunks])
15 print(f"Total chunks: {len(chunks)}")
16 print(f"Total vectors created: {len(vectors)}")
```

This embedding object will be used to convert all chunks into vectors.

SEMANTIC SIMILARITY IN VECTOR SPACE



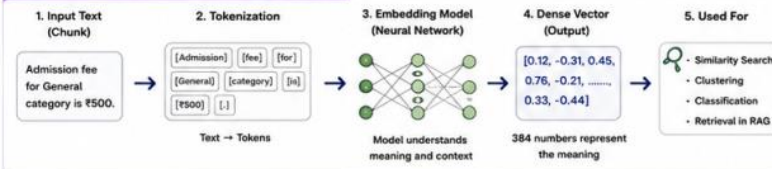
Key Points

- ★ Similar meaning → vectors are close.
- ★ Different meaning → vectors are far.
- ★ Search is performed by distance (cosine similarity).

Common Embedding Models

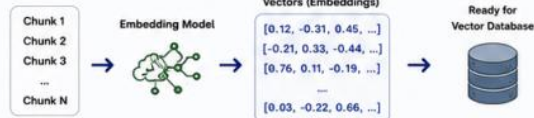
- all-MiniLM-L6-v2 (384 dimensions) ← Used here
- all-mpnet-base-v2 (768 dimensions)
- multi-qa-MiniLM-L6-cos-v1 (384)
- BGE-small-en-v1.5 (384)
- text-embedding-3-small (1536) (OpenAI)

HOW EMBEDDING WORKS (BEHIND THE SCENES)



OUTPUT OF THIS STEP

You now have a list of vectors (embeddings), one for each chunk. These vectors are ready to be stored in a Vector Database (FAISS / ChromaDB) in the next step.



IMPORTANT NOTES

- ✓ The embedding model does NOT generate answers.
- ✓ It only converts text into numbers.
- ✓ The quality of embeddings directly affects retrieval accuracy.
- ✓ Most models on HuggingFace are free to use locally.
- ✓ all-MiniLM-L6-v2 is fast, lightweight and works very well for RAG.

KEY TAKEAWAY



Embeddings convert text chunks into numerical representations of meaning, enabling machines to find relevant information even when the exact words are different. This is the heart of **semantic search** in RAG.

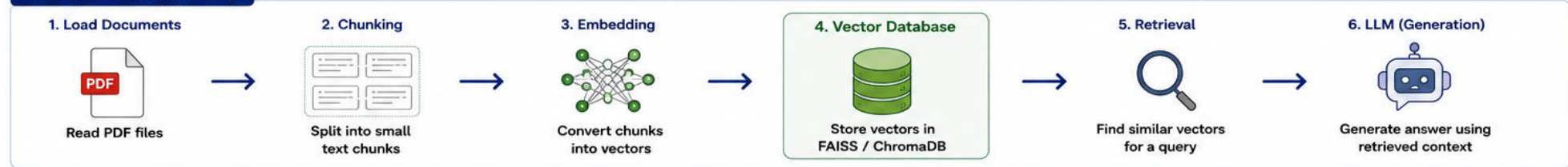
STEP 4: IMPLEMENTING THE VECTOR DATABASE

Store and Search Embeddings Efficiently



A Vector Database stores **embeddings (vectors)** and allows **fast similarity search**.
It is the **memory** of your RAG system.

WHERE WE ARE IN RAG PIPELINE



WHAT IS A VECTOR DATABASE?

- ✓ Stores high-dimensional vectors (embeddings).
- ✓ Indexes vectors for ultra-fast similarity search.
- ✓ Returns the most relevant vectors (chunks) for a query.
- ✓ Examples: FAISS, ChromaDB, Pinecone, Weaviate, Milvus.

FEATURES

- 🔍 High Performance Search (millions of vectors in ms)
- 📈 Scalability (handle large volumes easily)
- 🔗 Similarity Search (cosine, dot product, L2, etc.)
- 🏷️ Metadata Filtering (filter by source, page, tags, etc.)
- 💾 Persistence (save and load the index)
- 🔄 Incremental Updates (add/delete vectors)

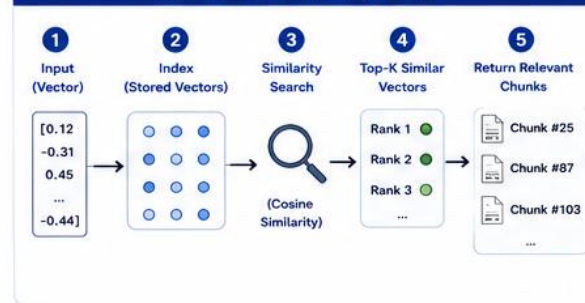
USES IN RAG

- 📄 Store embeddings of document chunks.
- 🔍 Given a user query, return top-k similar chunks.
- ⚙️ Provide relevant context to the LLM.
- 🛡️ Improve accuracy and reduce hallucinations.

COMMON VECTOR DATABASES

Database	Best For
FAISS	Local use, high performance, large datasets
ChromaDB	Easy to use, local & cloud, great for RAG
Pinecone	Managed cloud service, production ready
Weaviate	Open source, schema & metadata rich
Milvus	Enterprise scale, distributed

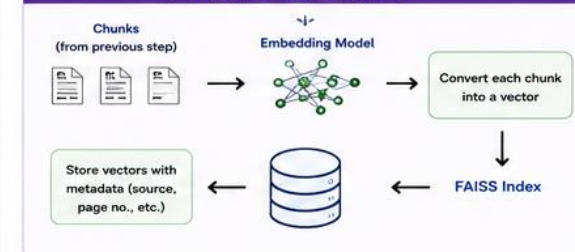
HOW VECTOR DATABASE WORKS



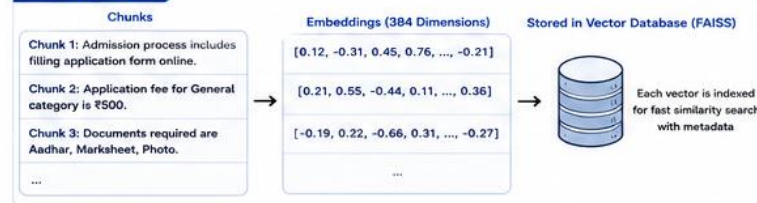
PYTHON CODE (Using LangChain + FAISS)

```
1 from langchain_community.vectorstores import FAISS
2 from langchain_huggingface import HuggingFaceEmbeddings
3
4 # 1. Load embedding model
5 embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
6
7 # 2. 'chunks' is the list of Document objects (from previous step)
8 # 'each chunk has .page_content and .metadata'
9
10 # 3. Create Vector Database (FAISS)
11 db = FAISS.from_documents(chunks, embedding)
12
13 # 4. Save the database locally
14 db.save_local("faiss_index")
15 print("Vector database created and saved!")
```

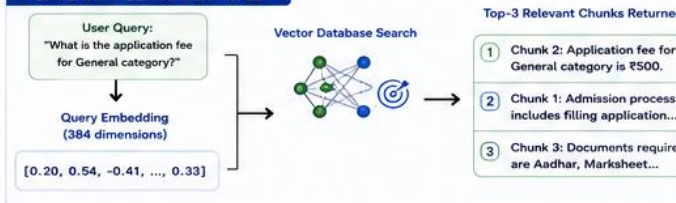
WHAT HAPPENS INTERNALLY?



EXAMPLE



SIMILARITY SEARCH EXAMPLE



KEY TAKEAWAY

- 🌟 Vector DB stores embeddings, not raw text.
- ✓ Enables lightning-fast semantic search.
- ✓ Returns most relevant chunks for any query.
- ✓ FAISS is best for local development.
- ✓ ChromaDB/Pinecone are great for production.
- ✓ This step is the heart of efficient retrieval in RAG.



Always persist your vector database (save index).
Rebuilding embeddings for large docs is time consuming.

Common Similarity Metrics



Cosine Similarity – measures angle between vectors (most common)



Dot Product – measures magnitude and direction



L2 Distance – measures straight line distance between vectors

STEP 5: RETRIEVER – FIND THE MOST RELEVANT CHUNKS



The Retriever takes a **user query**, converts it into a **vector (embedding)** and searches the Vector Database to find the **top-k most similar chunks**. These chunks are then used to provide context to the LLM.

WHERE WE ARE IN RAG PIPELINE



WHAT IS A RETRIEVER?

- ✓ Accepts a user query (natural language).
- ✓ Converts the query into an embedding.
- ✓ Searches the Vector Database for similar vectors.
- ✓ Returns top-k relevant document chunks.
- ✓ Acts as the memory search engine of RAG.

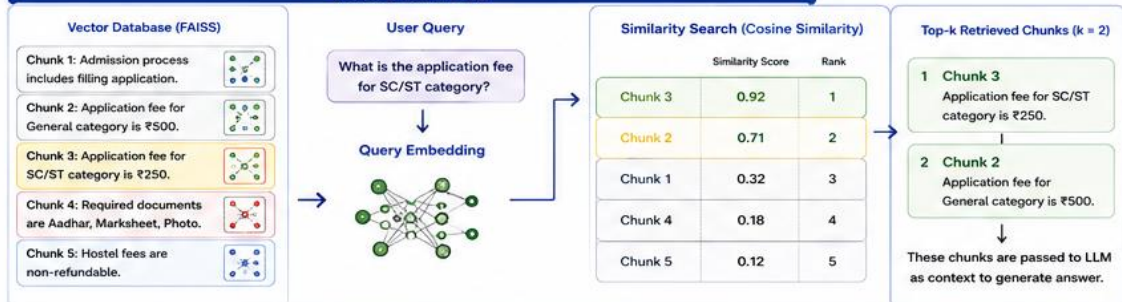
HOW RETRIEVER WORKS (IN DETAIL)

- 1 User asks a question (query).
- 2 Query is converted into an embedding using the same embedding model.
- 3 Embedding is compared with all stored vectors in the Vector DB.
- 4 Similarity score is computed (Cosine Similarity / L2 / Dot Product).
- 5 Top-k chunks with highest similarity are selected.
- 6 Retrieved chunks are passed to LLM as context.

PYTHON IMPLEMENTATION (LangChain + FAISS)

```
1 from langchain_community.vectorstores import FAISS
2 from langchain_huggingface import HuggingFaceEmbeddings
3
4 # 1. Load embedding model (same as used during indexing)
5 embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
6
7 # 2. Load existing FAISS vector database
8 db = FAISS.load_local("faiss_index", embedding, allow_dangerous_deserialization=True)
9
10 # 3. Create Retriever (searcher)
11 retriever = db.as_retriever(search_type="similarity", search_kwargs={"k": 3})
12
13 # 4. User query
14 query = "What is the application fee for SC/ST category?"
15
16 # 5. Retrieve top-k relevant chunks
17 docs = retriever.get_relevant_documents(query)
18
19 # 6. Print retrieved chunks
20 print(f"\nUser Query: {query}")
21 print("\nRetrieved Chunks:")
22 for i, doc in enumerate(docs):
23     print(f"\nChunk {i+1} (Score may vary):")
24     print(doc.page_content)
```

VISUAL EXAMPLE



ACTUAL OUTPUT (SAMPLE)

User Query: What is the application fee for SC/ST category?

Retrieved Chunks:

Chunk 1 (Score may vary):
Step 3: Pay the application fee online.
For SC/ST category the fee is Rs. 250. Fee is non-refundable.

Chunk 2 (Score may vary):
Step 3: Pay the application fee online.
For General category the fee is Rs. 500 and for SC/ST category the fee is Rs. 250.

KEY FEATURES OF RETRIEVER

- ✓ Fast similarity search over millions of vectors.
- ✓ Returns only the most relevant information.
- ✓ Improves accuracy and reduces hallucination.
- ✓ Configurable (top-k, search type, score threshold).
- ✓ Supports metadata filtering (source, page, date, etc.).

SEARCH TYPES IN LANGCHAIN

- 🔍 **similarity** → Standard similarity search (default)
- 🔍 **mmr** → Max Marginal Relevance (diverse results)
- 🔍 **similarity_score_threshold** → Returns results above a score threshold

USES OF RETRIEVER

- 🔍 Finds relevant information from large documents instantly.
- 🎯 Provides precise context to LLM for better answers.
- 🚀 Enables real-time Q&A over private data (RAG applications).
- 🛡️ Reduces incorrect answers by giving only relevant chunks.
- 💾 Works efficiently even with very large knowledge bases.

SUMMARY

💡 The Retriever is the heart of RAG search. It takes a question, searches the Vector Database for the most relevant chunks using embeddings, and returns them to the LLM as context.

Similarity Metrics Used

- 📐 **Cosine Similarity** (Most Common)
- 📐 **Dot Product** (Magnitude + Direction)
- 📐 **L2 Distance** (Euclidean Distance)

Best Practices

- ✓ Use appropriate chunk size (e.g., 300–800 chars).
- ✓ Use overlap in chunking to preserve context.
- ✓ Use top-k = 3 to 10 (depends on use case).
- ✓ Filter by metadata to improve precision.

STEP 6: PROMPT AUGMENTED LAYER – GIVE CONTEXT TO LLM



The Prompt Augmented Layer combines the user **query** with the **retrieved relevant chunks (context)** and sends it to the LLM. This helps the LLM generate accurate, **grounded** and hallucination-free answers.

WHERE WE ARE IN RAG PIPELINE



WHAT IS PROMPT AUGMENTATION?

- ✓ Takes the user question and the top retrieved chunks.
- ✓ Formats them into a well-structured prompt.
- ✓ Provides the LLM with the right context.
- ✓ Improves accuracy and reduces hallucinations.
- ✓ Also called **Context Augmentation**.

HOW PROMPT AUGMENTED LAYER WORKS

- 1 User asks a question.
- 2 Retriever returns top-k relevant chunks.
- 3 Chunks are inserted into a prompt template.
- 4 The augmented prompt is sent to the LLM.
- 5 LLM generates answer based on the given context.

TYPICAL PROMPT TEMPLATE

You are a helpful assistant for a college website.
Use the following context to answer the question.
If the answer is not in the context, say "I don't know".

Context:
{context}

Question:
{question}

Answer:

Instruction to the LLM
(behavior + rule)

Retrieved chunks
will be placed here

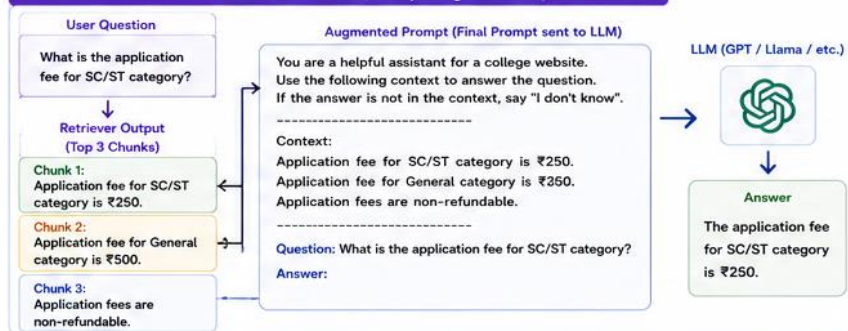
User question
will be placed here

LLM will generate
final answer here

PYTHON IMPLEMENTATION (LangChain)

```
1 from langchain.prompts import PromptTemplate
2 from langchain.chains import RetrievalQA
3
4 # 1. Define Prompt Template
5 template = """You are a helpful assistant for a college website.
6 Use the following context to answer the question.
7 If the answer is not in the context, say "I don't know".
8
9 Context:\n{context}\n\n
10 Question: {question}\n\n";
11 Answer:"""
12
13 prompt = PromptTemplate(
14     input_variables=["context", "question"],
15     template=template
16 )
17
18 # 2. Create RetrievalQA chain (uses retriever + LLM)
19 qa = RetrievalQA.from_chain_type(
20     llm=llm, # your LLM (e.g., OpenAI, Groq, etc.)
21     chain_type="stuff", # stuff => put all chunks in prompt
22     retriever=retriever, # from previous step (FAISS retriever)
23     chain_type_kwargs={"prompt": prompt}
24 )
25
26 # 3. Ask a question
27 question = "What is the application fee for SC/ST category?"
28 result = qa.run(question)
29 print(result)
```

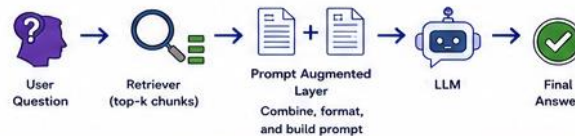
VISUAL EXAMPLE (Prompt Augmentation)



USES / BENEFITS



BEHIND THE SCENES FLOW



BEST PRACTICES

- ✓ Use a clear and strong system instruction.
- ✓ Include only top-k most relevant chunks.
- ✓ Limit total tokens in prompt (avoid very long context).
- ✓ Add metadata (source, page no.) if needed.
- ✓ Always tell the model: "If answer not in context, just say I don't know."



KEY TAKEAWAY

The Prompt Augmented Layer is the bridge between retrieval and generation. It gives the LLM the right context in the right format to produce accurate answers.



IMPORTANT TIP

Good prompt + Good context (chunks) = High quality, trustworthy answers.

STEP 7: LLM GENERATOR – GENERATE THE FINAL ANSWER



The LLM Generator takes the augmented prompt (question + retrieved context) and generates a natural, accurate and context-aware answer. This is the **final step of the RAG pipeline**.

WHERE WE ARE IN RAG PIPELINE



WHAT IS LLM GENERATOR?

- ✓ Takes the augmented prompt as input.
- ✓ Uses its knowledge + provided context to generate the answer.
- ✓ Produces human-like, accurate and well-structured responses.
- ✓ Can also provide citations / sources.
- ✓ Different LLMs: OpenAI (GPT), Google Gemini, Llama, Mistral, etc.

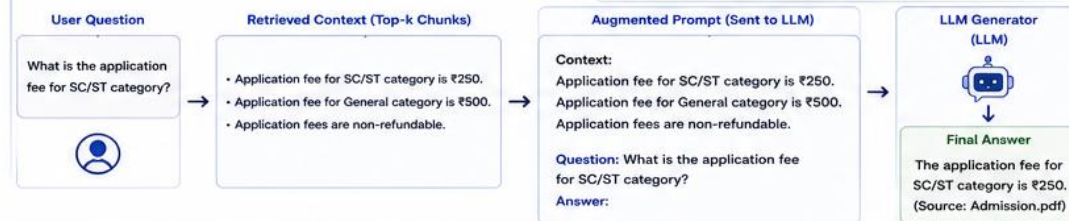
HOW LLM GENERATOR WORKS

1. Receives the augmented prompt from previous step.
2. Understands the question and reads the provided context.
3. Generates an answer only based on the given context (plus model knowledge if allowed).
4. Formats the response in a clean and readable way.
5. Returns the final answer to the user.

PYTHON IMPLEMENTATION (LangChain + OpenAI / Gemini)

```
1 # 1. Import LLM (Example with OpenAI Chat Model)
2 from langchain_openai import ChatOpenAI
3 from langchain_core.messages import HumanMessage, SystemMessage
4
5 # 2. Initialize the LLM
6 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2) # or "gpt-4", "gemini-pro"
7
8 # 3. Augmented prompt (from previous step)
9 system_prompt = """You are a helpful assistant.
10 Answer the question using the given context only.
11 If the answer is not in the context, say "I don't know".
12 Provide source information if possible."""
13
14 user_prompt = f"""
15 Context:
16 {context}
17 Question: {question}
18 Answer:"""
19
20 # 4. Generate answer
21 response = llm.invoke([
22     SystemMessage(content=system_prompt),
23     HumanMessage(content=user_prompt)
24 ])
25
26 # 5. Print final answer
27 print("\nFINAL ANSWER:")
28 print(response.content)
```

EXAMPLE: LLM GENERATION



COMPLETE CODE EXAMPLE (RAG – LAST STEP)

```
1 from langchain_openai import ChatOpenAI
2 from langchain_core.messages import SystemMessage, HumanMessage
3
4 # Assume 'context' contains retrieved chunks text and 'question' is user query
5 context = """Application fee for SC/ST category is ₹250.
6 Application fee for General category is ₹500.
7 Application fees are non-refundable."""
8 question = "What is the application fee for SC/ST category?"
9
10 system_prompt = """You are a helpful assistant.
11 Answer the question using the given context only.
12 If the answer is not in the context, say "I don't know".
13 Provide source information if possible."""
14
15 user_prompt = f"""
16 Context:
17 {context}
18 Question: {question}
19 Answer:"""
20
21 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
22 response = llm.invoke([
23     SystemMessage(content=system_prompt),
24     HumanMessage(content=user_prompt)
25 ])
26 print("\nFINAL ANSWER:")
27 print(response.content)
```

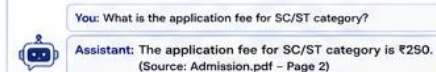
BEST PRACTICES

- ✓ Use clear system instructions.
- ✓ Limit context length to avoid token overflow.
- ✓ Provide only relevant context (top-k chunks).
- ✓ Use citations / sources for trust.
- ✓ Set temperature low (0.1 – 0.3) for factual answers.
- ✓ Handle "I don't know" cases properly.

THINGS TO REMEMBER

- 💡 LLM does not search the documents itself. It only generates answers using the context you provide. Better context = Better answers.

FINAL OUTPUT SAMPLE (USER VIEW)



KEY USES

- ✓ Answers user queries in natural language.
- ✓ Generates summaries, explanations, comparisons.
- ✓ Provides step-by-step solutions.
- ✓ Can cite sources for transparency.
- ✓ Supports multi-turn conversations.

LLMs YOU CAN USE

- OpenAI (GPT-4o / GPT-4 / GPT-3.5)
- Google (Gemini Pro / Gemini 1.5)
- Meta (Llama 3)
- Mistral AI
- Anthropic (Claude)
- Open Source Models (Mixtral, Zephyr, etc.)

SUMMARY

- 🏆 The LLM Generator is the brain that converts retrieved context into meaningful, human-like answers for the user.